HazBot: Architecture and Coordination Verification of A Raspberry Pi-Based Mobile Manipulator for Hazardous-Object Retrieval

Anonymous Authors

***Abstract*—Retrieving an object from a potentially hazardous area requires coordinated perception, mobility and manipulation. We present HazBot, a modular mobile-manipulation architecture centered on a Raspberry Pi with hazardous-object retrieval as its application objective. Four subsystems provide LiDAR navigation, differential wheel drive, camera-based onboard inference and an articulated gripper. The camera connects directly to the Pi and dedicated Arduino controllers serve the wheels and arm, while the LiDAR retains its sensor-specific controller. We formalize a seven-phase retrieval mission and the evidence required at each subsystem handoff. A ball-based nonhazardous surrogate motivates the task. The evaluation uses an executable discrete coordination model, source-isolated command analysis and URDF-derived geometry. A full analysis of 261,121 wheel-command pairs shows direction-only magnitude reduction and 1,020 mixed-zero inputs that don't send any new commands. We got a shoulder-to-wrist bound of 0.200 m from the URDF link lengths, and the modeled geometric envelope is made up of 68,921 joint samples. The full reference coordinator admits no command-contract violations across 28,672 phase–observation pairs. These results connect the mission design to concrete interface and reachability constraints. The contribution is a reproducible architecture and coordination study that supports task-specific recognition and subsequent application validation.**

***Index Terms*—Mobile manipulation, Raspberry Pi, hazardous-object retrieval, embedded vision, subsystem coordination, model-based verification.**

I. Introduction

Moving a suspect object away from people requires locating it, navigating toward it, positioning a gripper, transporting it and confirming release. Components can function in isolation while the complete task remains unreliable: detections become stale, gateways discard command magnitude, and navigation goals may lie outside the arm's contact workspace. The central engineering problem is the relationship between subsystem evidence and permission to act.

Mobile robots support nuclear decommissioning [1], while compact manipulators integrate mobility and reach [2]. HazBot studies an accessible embedded implementation with four Pi-centered subsystems: navigation, wheel control, native visual inference and manipulation. Dedicated controllers separate sensor acquisition and actuator signaling from mission decisions.

The application is retrieval of designated objects from a potentially hazardous area. Visual recognition proposes a candidate; task authority determines whether it may be handled and where it should be deposited. Appearance alone does not establish chemical, radiological or biological properties. Ball-based simulated control supplies the nonhazardous surrogate context.

The paper makes three contributions. First, it specifies a Pi-centered four-subsystem architecture (Fig. 1) with explicit physical interfaces and separate drive, arm and ranging controller paths. Second, it defines a seven-phase reference coordinator with observable prerequisites for mobility–manipulation handoffs. Third, it provides reproducible model and source analyses that connect these prerequisites to wrist geometry and actual command semantics. The evaluation establishes properties of the discrete coordinator and source models which provides a traceable basis for subsequent physical retrieval experiments.

II. Related Work

A. Application and Mobile Manipulation

Vega provides a relevant hazardous-environment precedent: Bird *et al.* describe a small ground robot for nuclear decommissioning with engineering directed toward its operating environment [1]. Its application illustrates the importance of environment-specific qualification. Stretch combines a differential mobile base and compact manipulator, supported by design analysis and physical task demonstrations [2]. HomeRobot treats navigation, recognition, grasping and placement as a connected mobile-manipulation problem with distinct simulation and real-world evaluation components [3]. These studies motivate HazBot's complete retrieval sequence and the coordination of its sensing and actuation modules.

B. Embedded Integration and Coordination

ROS 2 provides a common communication and software framework for distributed robotic components [4]. Nav2 [5] and SLAM Toolbox [6] provide established navigation and mapping capabilities. These components supply reusable interfaces for integrating onboard observations and motion requests. Edge-device detector evaluations further motivate explicit characterization of model and device choices [7], including the configuration used for local inference.

Command arbitration and sensor-dependent stopping are established in systems such as `twist_mux` and Nav2 Collision Monitor [8], [9]. HazBot focuses on the connection between a four-subsystem embedded design, retrieval-phase prerequisites and reproducible characterization of the particular geometry and gateways that implement those interfaces. The reference

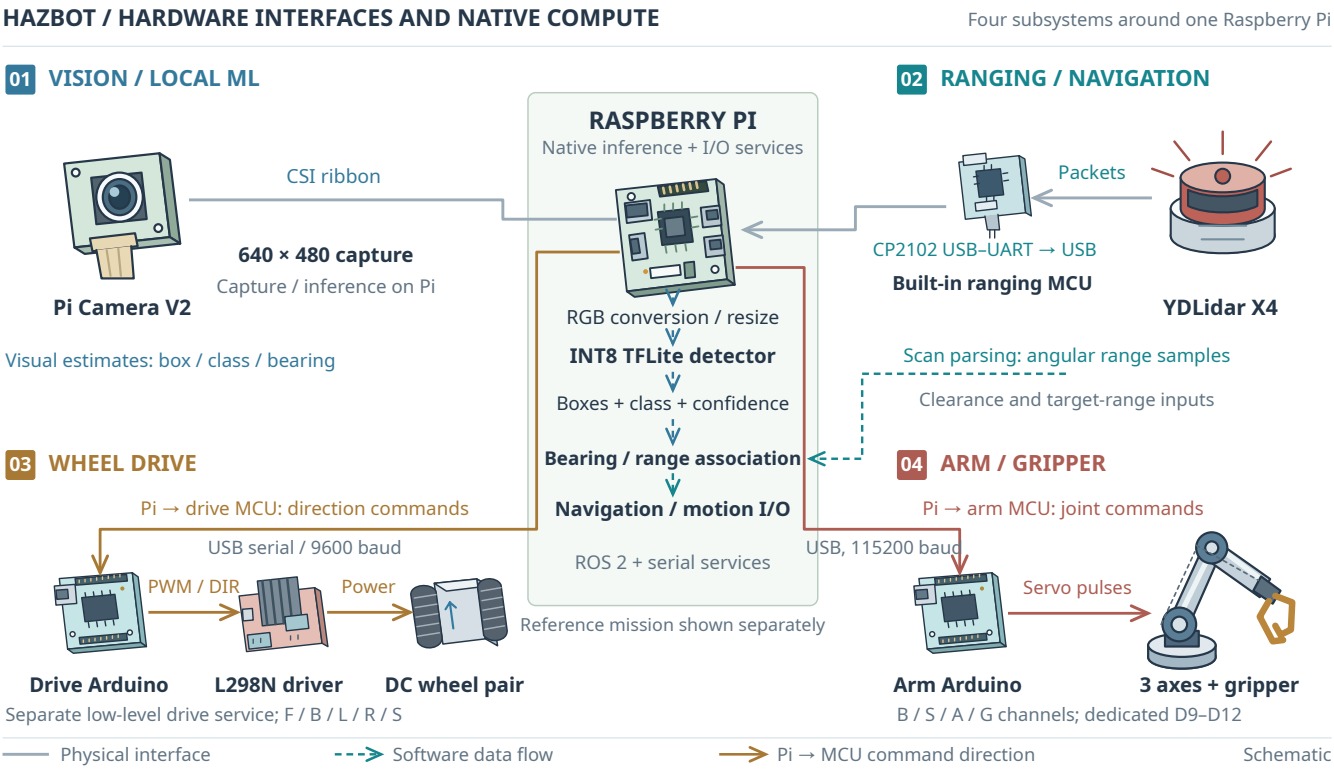


Fig. 1. HazBot's four-subsystem architecture. The Raspberry Pi performs onboard computation and interfaces with a direct CSI camera, a LiDAR sensor controller, a dedicated drive Arduino, and a separate arm Arduino. Solid paths identify physical interfaces and dashed paths show software data flow. Glyphs identify functional hardware rather than a scale assembly. The complete mission coordinator is evaluated as a reference model.

TABLE I
FOUR SUBSYSTEMS AND THEIR PI INTERFACES

Subsystem	Connection and responsibility
Vision / ML	CSI camera directly to Pi; local TFLite inference.
Navigation	LiDAR sensor controller and USB–UART path; ranging data consumed on Pi.
Wheel drive	USB serial, 9600 baud, to drive Arduino; PWM/direction through L298N.
Manipulator	Separate USB serial, 115200 baud, to arm Arduino; four servo channels.

coordinator makes this integration inspectable before a physical mission benchmark is attempted.

III. System Architecture

A. Central Computer and Physical Interfaces

Figure 1 shows the hardware interfaces; Fig. 2 connects sensing and Pi processing to command generation through the reference coordinator. The analyzed configuration comprises a Pi Camera V2, YDLIDAR X4, L298N differential drive and servo arm; Table I lists the interfaces. A workstation supports visualization without hosting the native detector. Figures 3 and 4 show the prototype and laptop; the local display uses GPIO and the camera a white CSI ribbon. Figure 5 documents the command surfaces, not live-system performance.

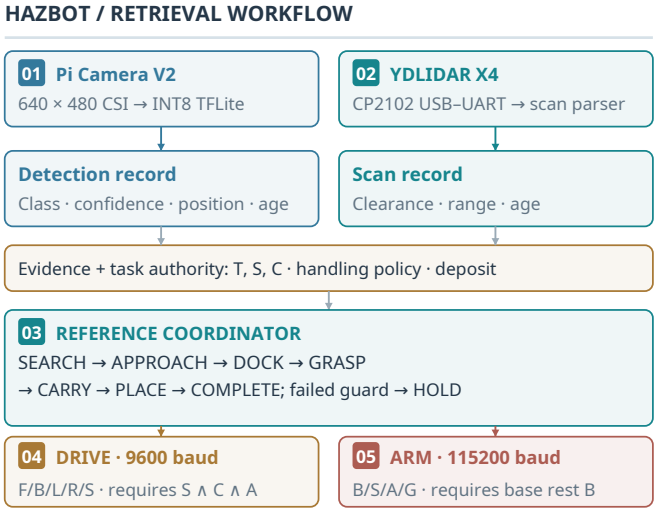


Fig. 2. Retrieval workflow: observations and task authority feed the reference coordinator, then dedicated drive and arm controllers. Guards follow Eqs. 2–4. Base-rest, grip and release feedback remains to be instrumented.

B. Navigation and Wheel Control

The LiDAR controller acquires ranges and sends packets to the Pi over USB. Host navigation uses scans for route clearance and target-distance association. ROS 2 mapping

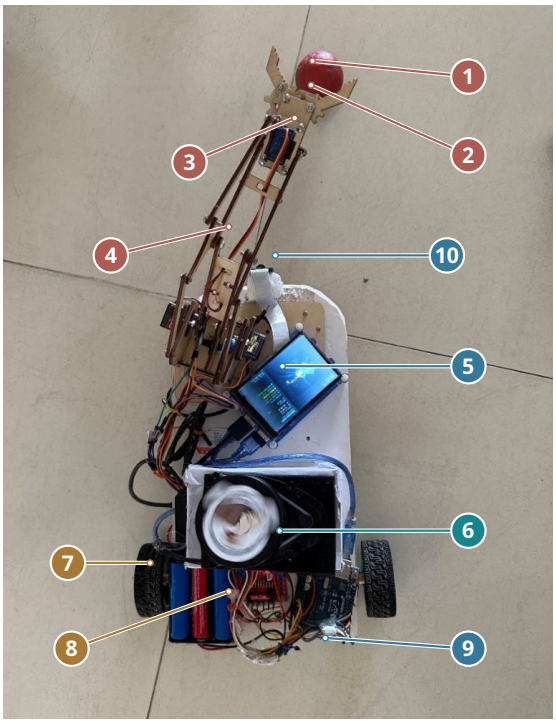


Fig. 3. Physical HazBot prototype. Markers identify: 1, ball surrogate; 2, gripper; 3, servo; 4, arm linkage; 5, Raspberry Pi with GPIO-mounted display; 6, LiDAR; 7, drive wheel; 8, battery cells; 9, control electronics; 10, camera at the end of the white CSI ribbon.

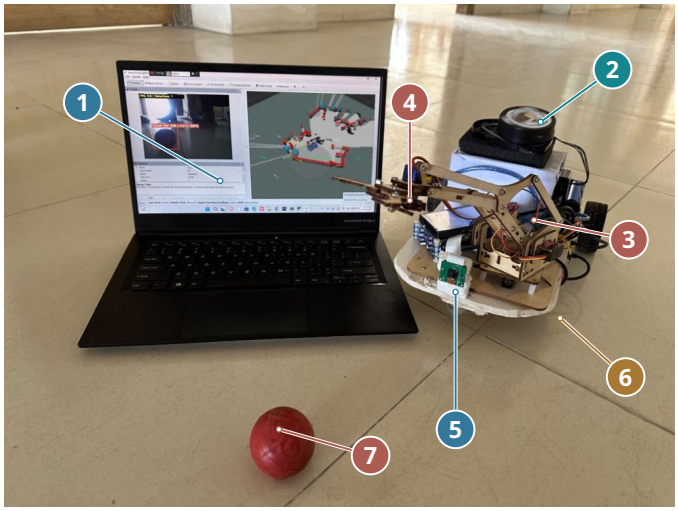


Fig. 4. Front view with laptop visualization. Markers identify: 1, laptop visualization; 2, LiDAR; 3, arm linkage; 4, gripper; 5, CSI camera; 6, mobile base; 7, ball surrogate.

components are available, but map quality and localization error are not measured here.

The wheel Arduino drives the motor bridge independently of the arm controller. For an ideal differential base, wheel radius R and track b relate desired body velocity (v, ω) to wheel angular rates:

$$\dot{\phi}_L = \frac{v - b\omega/2}{R}, \qquad \dot{\phi}_R = \frac{v + b\omega/2}{R}. \tag{1}$$

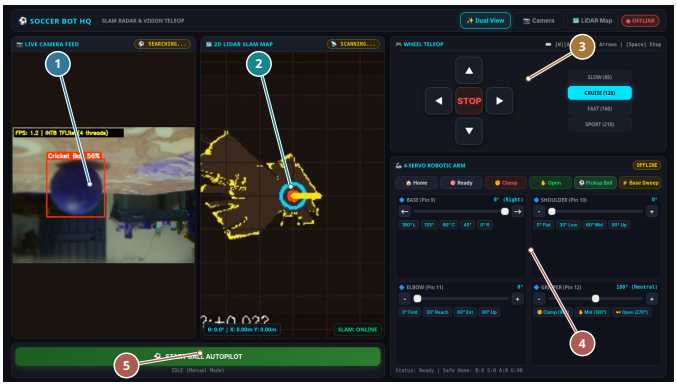


Fig. 5. Annotated operator dashboard supplied with the prototype. Markers identify: 1. camera view and ball-detection panel; 2. LiDAR/SLAM map; 3. wheel teleoperation controls; 4. four-servo arm controls; 5. ball-autopilot action. The screenshot documents the intended monitoring and command surfaces; it is not evidence of online connectivity, detector accuracy, map quality or autonomous mission success.

The model uses $R = 0.033\,\mathrm{m}$ and $b = 0.210\,\mathrm{m}$. Realizing Eq. 1 requires a gateway that preserves wheel-rate semantics; the command study tests this condition.

C. Vision and Native Inference

The direct CSI path configures 640×480 RGB capture and a two-thread TFLite interpreter for an INT8 detector. Its tennis- and cricket-ball labels provide task surrogates. Hazard-related classes require a corresponding dataset and evaluation. Model weights are absent from the analyzed artifacts; accuracy and execution time are not reported.

A detection record should carry class, confidence, image position and age. Metric approach also requires range associated with the same target and frame. Class acceptance and freshness are distinct: a confident stale record cannot authorize approach. Task authority supplies handling permission; confidence establishes neither material identity nor authorization.

D. Robotic Hand and Arm

The arm has base yaw, shoulder pitch and elbow pitch, a fixed wrist and a separately actuated gripper. Its dedicated Arduino drives four servo channels, not four positioning joints. Existing commands include a preset pickup sequence; the seven-phase coordinator is a separate executable reference model. Separate drive and arm boards retain distinct serial protocols and pin assignments.

IV. Retrieval Coordination and Analysis Method

Figure 6 links the mission specification, URDF chain and actuator gateways to three offline studies: coordinator verification, wrist geometry and command semantics. Their evidence identifies integration requirements for subsequent physical trials.

A. Mission and Evidence Contracts

The reference mission has seven phases (Fig. 7): SEARCH authorizes a candidate; APPROACH reaches its pickup region;

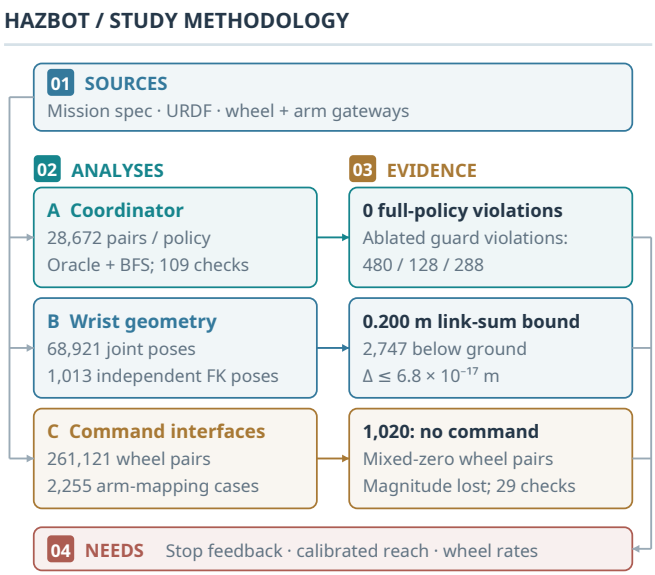


Fig. 6. Study methodology: four source artifacts feed three offline analyses and their integration findings. The studies execute the reference model, source-isolated command logic and URDF geometry; they do not operate the robot, ROS nodes, serial hardware or detector.

DOCK transfers control from base to arm; GRASP secures the object; CARRY transports it with the arm stowed; PLACE releases it at the deposit region; and COMPLETE requires a release indication.

Let T denote an authorized, fresh target; S a fresh, usable scan; C a clear route; A a stowed arm and B a stopped base. The principal command contracts are

$$\text{base motion} \Rightarrow S \wedge C \wedge A, \tag{2}$$

$$\text{target approach} \Rightarrow T, \tag{3}$$

$$\text{arm motion} \Rightarrow B. \tag{4}$$

Reach and closure additionally require T and wrist reachability; release requires the deposit region and a grip indication. The controller also requires scan health during docking, grasping and placement. A failed prerequisite emits HOLD without advancement. HOLD inhibits active motion and needs an explicit actuator-boundary stop. Target acquisition and release may advance phase while emitting HOLD. A stop request does not establish B: deployment requires measured base-rest feedback and a defined confirmation rule.

The abstract observations comprise twelve Boolean values: target authorization and freshness; scan freshness and usability; route clearance; arm stowage; base stopped; wrist reachability; pickup-region arrival; grip confirmation; deposit-region arrival and release confirmation. Each has a specific role in the interface but this discrete model does not generate these observations from physical sensors. In particular, grip and release indications are input assumptions, not inferred force or contact measurements.

B. Bounded Reference-Model Verification

The verification procedure in Fig. 8 enumerates every observation valuation at every phase: $7 \times 2^{12} = 28{,}672$ phase–observation pairs per policy. No valuations are pruned. A separate admissibility oracle checks the output action against Eqs. 2–4 and the target, reach and placement conditions. Breadth-first exploration establishes phase reachability and extracts shortest-prefix counterexamples. Three ablations remove the base–arm interlock, target authorization or scan-age requirement from the controller while retaining the full oracle. This design tests whether each omitted condition creates a concrete contract violation.

All Boolean valuations are retained, including physically inconsistent combinations. Counts therefore describe the specification domain, not robot failure probabilities. A nominal trace checks progression to COMPLETE under admissible observations; neither it nor existential phase reachability establishes physical mission success.

C. Source-Derived Wrist Geometry

Figure 9 connects arm geometry to the docking prerequisite. From the URDF chain, the shoulder lies at $(0.100, 0, 0.155)$ m in the base frame and both positioning links have length $L_1 = L_2 = 0.100$ m. The zero pose has a vertical upper link and horizontal forearm. The figure uses $q_1 = q_b$, $q_2 = q_s$, and $q_3 = q_e$. For yaw q_b, shoulder angle q_s, and elbow angle q_e, the wrist origin is

$$r = L_1 \sin q_s + L_2 \cos(q_s + q_e), \tag{5}$$

$$x = 0.100 + r \cos q_b, \quad y = r \sin q_b, \tag{6}$$

$$z = 0.155 + L_1 \cos q_s - L_2 \sin(q_s + q_e). \tag{7}$$

We sample 41 values, including both bounds, for each positioning joint. The declared bounds are $q_b \in [-1.5708, 1.5708]$, $q_s \in [-0.5, 2.6179]$, and $q_e \in [-1.5708, 2.6179]$ rad. An independent homogeneous-transform implementation evaluates the URDF chain at 1,000 seeded random poses and 13 special poses. Agreement checks numerical implementation, while the grid characterizes the ideal wrist envelope. The wrist origin is not a calibrated grasp contact point; collision, load, compliance and stability constraints are not represented.

D. Command-Interface Characterization

Using Python's abstract syntax tree, we isolate the `SET:` branch of the actual wheel gateway and replace only the transport sender with an in-memory recorder. All integer pairs $(l, r) \in [-255, 255]^2$ are executed and compared with an independently specified sign partition. The arm gateway's pure angle mapper is evaluated for five accepted aliases over 451 inputs from -90 to 360, checking clamps and reversals. The elbow has two aliases, so this represents four actuator channels. No ROS constructors, sockets, motor servers or hardware launchers are executed. Source hashes, settings, CSV outputs and validation records bind the results to the analyzed implementation.

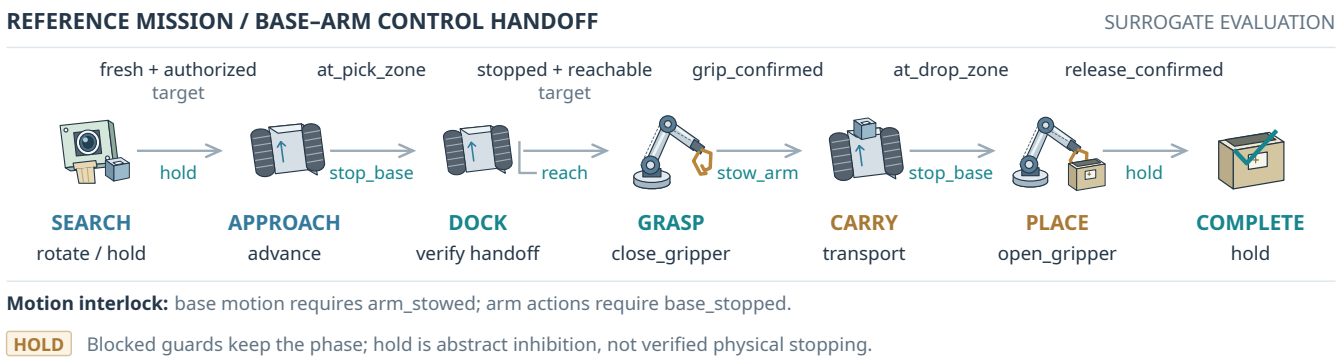


Fig. 7. Seven-phase reference retrieval mission. Conditions summarize the handoff evidence; complete command predicates are defined in the text and executable model. A failed prerequisite requests HOLD at the current phase. The strip depicts the coordination specification, rather than a measured physical trajectory.

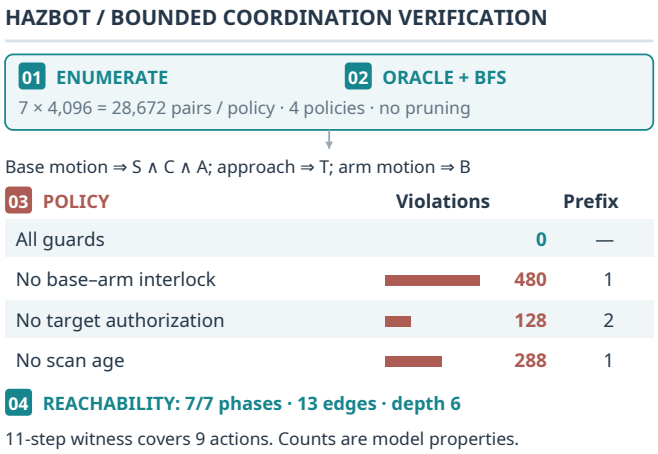


Fig. 8. Bounded verification: all phase–observation pairs are checked against an independent oracle. Prefix length counts observations to the first violating output. Reachability concerns the full policy; counts describe model properties, not robot failure rates.

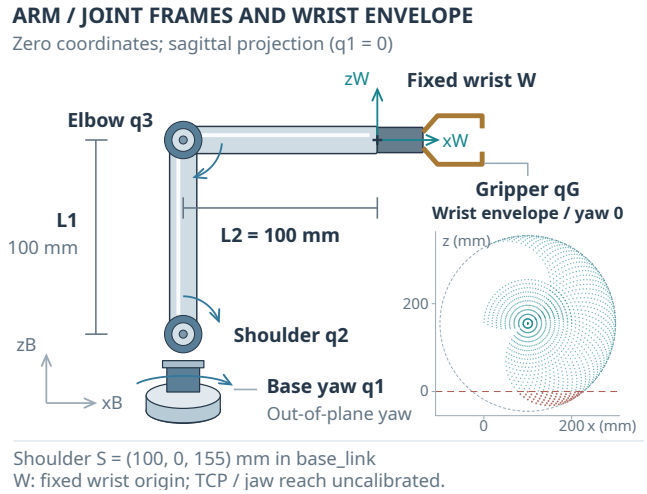


Fig. 9. Source-derived arm geometry at zero joint coordinates, with joint axes and fixed wrist origin. The small inset shows 1,681 zero-yaw wrist samples, the link-sum bound, and modeled ground exclusions. Samples describe ideal geometry rather than calibrated contact positions or collision-free grasps.

V. Results

A. Mission Handoffs

Figure 8 combines the verification procedure, ablation counts and phase-reachability results. The full reference coordinator admits 0 contract-violating commands among 28,672 phase–observation pairs. All 7 phases are reachable with 13 unique phase edges (including self-loops) and a shortest completion depth of 6 transitions. A separate 11-step nominal witness exercises all 9 action types and reaches COMPLETE.

Removing the base–arm interlock produces 480 violating pairs; removing target authorization produces 128 and removing scan-age checks produces 288. Their shortest counterexample prefixes contain 1, 2, and 1 observations respectively. For example, the interlock ablation permits SEARCH rotation while the arm is not stowed. These counts demonstrate guard sensitivity over the enumerated domain and they are not observed mission failure rates.

B. Geometric Envelope and Docking

The 41^3 grid contains 68,921 wrist poses with a link-sum bound of 0.200 m. Sampled height spans -0.0316 to 0.3549 m: 2,747 poses lie below modeled ground and 66,174 on or above it. These are joint-grid counts, not Cartesian volumes; above-ground poses may still violate collision or stability constraints.

Independent forward kinematics agrees over 1,013 poses within 6.8×10^{-17} m, validating numerical implementation rather than mechanical accuracy. Docking requires calibrated contact geometry and target pose; the ideal wrist envelope is a starting point for that calibration.

C. Wheel and Arm Command Semantics

All 261,121 wheel pairs match the sign partition. Each nonzero quadrant maps 65,025 pairs to one direction command, discarding magnitude. Only $(0, 0)$ sends stop;

1,020 mixed-zero pairs send nothing. Thus, a stationary-wheel/moving-wheel request is not preserved. No new command is distinct from stop; this study observes messages, not subsequent actuator behavior.

This qualifies Eq. 1: approach and docking require deliberate discrete primitives or a magnitude-preserving, calibrated wheel-rate path. No physical speed error is measured.

All 2,255 arm cases match the independent clamp/reversal specification. Base direction is retained, shoulder and elbow conventions reversed, and the gripper clamped to its range. Mapping commands to modeled angles still requires calibration. All 29 geometry/interface checks pass. The two detector labels confirm only the surrogate configuration, not recognition accuracy.

VI. Discussion and Application Development

The architecture connects object retrieval through explicit subsystem evidence: vision proposes a candidate, task authority permits handling, navigation establishes a pickup region, the base confirms rest and manipulation confirms grip and release. The mission model exposes these obligations where hardware feedback remains to be developed. Its nominal trace demonstrates logical progress under stated assumptions, while source analysis identifies what the actuator gateway can express.

The next application step is to instantiate these observations with measured quantities. This requires a task-specific visual dataset and held-out recognition results; calibrated camera–LiDAR–arm frames and contact geometry; feedback for base rest, grip and release as well as timed physical runs covering navigation, pickup, transport, and deposit. Sensor-age limits must follow measured acquisition and transport behavior. Force, payload, containment and environmental compatibility must be evaluated for the designated object class. A general RGB category detector cannot establish an unknown material's hazardous properties.

The results establish source and model properties; application trials are necessary for physical obstacle avoidance and hazardous-environment suitability. Nonhazardous surrogates support the initial benchmark, followed by object and environment-specific qualification as illustrated by prior systems [1], [2].

VII. Conclusion

HazBot connects navigation, wheel control, native visual inference, and manipulation through a Raspberry Pi-centered architecture. Its seven-phase reference mission formalizes retrieval handoffs, while reproducible geometry and gateway analysis expose docking constraints, magnitude reduction, and mixed-zero command omissions. These artifacts provide an implementation-grounded basis for task-specific hazardous-object retrieval and subsequent physical evaluation.

Acknowledgment

The authors thank the ROS 2, Nav2 and open-source robotics communities for their publicly available software and documentation. ChatGPT, OpenAI's large language model, assisted manuscript preparation.

References

[1] B. Bird, M. Nancekievill, A. West, J. Hayman, C. Ballard, W. Jones, S. Ross, T. Wild, T. Scott, and B. Lennox, "Vega—a small, low cost, ground robot for nuclear decommissioning," *Journal of Field Robotics*, vol. 39, no. 3, pp. 232–245, 2022. [Online]. Available: https://onlinelibrary.wiley.com/doi/full/10.1002/rob.22048

[2] C. C. Kemp, A. Edsinger, H. M. Clever, and B. Matulevich, "The design of Stretch: A compact, lightweight mobile manipulator for indoor human environments," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3150–3157. [Online]. Available: https://arxiv.org/abs/2109.10892

[3] S. Yenamandra, A. Ramachandran, K. Yadav, A. S. Wang, M. Khanna, T. Gervet, T.-Y. Yang, V. Jain, A. Clegg, J. M. Turner, Z. Kira, M. Savva, A. X. Chang, D. S. Chaplot, D. Batra, R. Mottaghi, Y. Bisk, and C. Paxton, "HomeRobot: Open-vocabulary mobile manipulation," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 1975–2011. [Online]. Available: https://proceedings.mlr.press/v229/yenamandra23a.html

[4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: https://arxiv.org/abs/2211.07752

[5] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725. [Online]. Available: https://arxiv.org/abs/2003.00368

[6] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021. [Online]. Available: https://joss.theoj.org/papers/10.21105/joss.02783

[7] D. K. Alqahtani, M. A. Cheema, M. A. Rodriguez, and A. N. Toosi, "A comprehensive evaluation of deep learning object detection models on heterogeneous edge devices," arXiv:2409.16808v3, 2026, version 3, July 14, 2026. [Online]. Available: https://arxiv.org/abs/2409.16808v3

[8] ROS Teleop Contributors, "twist_mux: Input topic configuration," Official project source, rolling branch, accessed: Oct. 21, 2026. [Online]. Available: https://github.com/ros-teleop/twist_mux/blob/rolling/config/twist_mux_topics.yaml

[9] Nav2 Project, "Collision monitor node," Nav2 Documentation, accessed: Oct. 21, 2026. [Online]. Available: https://docs.nav2.org/rolling/configuration_and_development/configuration_guide/core_servers/collision_monitor/configuring_collision_monitor_node/